

Révisions

Partie I – Programmation en Python

I.1 – Types de bases

▷ **Question 1.** Quelles instructions calculent $\lfloor n/2 \rfloor$? (A) $n/2$ (B) $n//2$ (C) `int(n/2)`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 2.** Quelle instruction calcule le reste de la division euclidienne de a par b ?

(A) `a % b` (B) `a mod b` (C) `mod(a, b)`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 3.** Quelle instruction calcule x^n ? (A) `x^n` (B) `x^~n` (C) `x**n`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 4.** Quelle instruction calcule « b_1 et b_2 », où b_1 et b_2 sont des expressions booléennes ?

(A) `b_1 & b_2` (B) `b_1 && b_2` (C) `b_1 and b_2`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 5.** En quoi les évaluations de `b_1 and b_2` et `b_1 or b_2` sont paresseuses ?

(A) Python n'évalue pas `b_1` si l'évaluation de `b_2` permet de conclure.

(B) Python n'évalue pas `b_2` si l'évaluation de `b_1` permet de conclure.

(C) Python n'évalue toujours qu'un seul terme parmi `b_1` et `b_2`.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 6.** Quelle instruction calcule « $x \leq y$ » ? (A) `x <= y` (B) `x <= y`

Réponse(s) : ☐ (A) ☐ (B)

▷ **Question 7.** Quelle instruction calcule « $x \neq y$ » ?

(A) `x != y` (B) `x != y` (C) `x != y` (D) `x <> y` (E) `x not y`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

I.2 – Types structurés

▷ **Question 8.** Que vaut `[x//2 for x in L]` lorsque `L = [1, 2, 3, 4, 6]` ?

- (A) `[0, 1, 1, 2, 3]` (B) `[0, 1, 2, 3]` (C) `[0, 2]` (D) `[1, 2, 3]`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

▷ **Question 9.** Quelle est la complexité du test d'appartenance `x in L`, où `L` est une liste ?

- (A) $O(1)$. (B) $O(\log n)$ avec $n = \text{len}(L)$. (C) $O(n)$ avec $n = \text{len}(L)$.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 10.** Que vaut `L[0:2]` lorsque `L = [1, 2, 3, 4, 6]` ?

- (A) `[1]` (B) `[1, 2]` (C) `[1, 2, 3]` (D) `[1, 3, 6]`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

▷ **Question 11.** Quelle instruction crée un dictionnaire ayant une clé "a" associée à la valeur 0 ?

- (A) `{"a" = 0}` (B) `{"a" <- 0}` (C) `{"a" : 0}` (D) `{"a", 0}`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

▷ **Question 12.** Quelle est la complexité du test d'appartenance `x in D`, où `D` est un dictionnaire ?

- (A) $O(1)$ en moyenne. (B) $O(\log n)$ avec $n = \text{len}(D)$. (C) $O(n)$ avec $n = \text{len}(D)$.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

I.3 – Structures de contrôle

▷ **Question 13.** Quel programme assigne 7 à la variable `y` dans le cas où `x` vaut 3 ?

- (A) `if x = 3:
 y = 7`      (B) `if x == 3:
 y == 7`      (C) `if x == 3:
 y = 7`      (D) `if x = 3:
 y == 7`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

▷ **Question 14.** Soit `t = (8, "w")`. Quelles instructions permettent d'accéder au contenu de `t` ?

- (A) `x = t[0]` et `y = t[1]` (B) `x, y = t` (C) `(x, y) = t` (D) `t = x, y` (E) `t = (x, y)`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

▷ **Question 15.** Quels programmes affichent le contenu de la liste L ?

- (A) `for x in L:
 print(x)`      (B) `for i in range(len(L)-1):
 print(L[i])`      (C) `for i in range(len(L)):
 print(L[i])`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 16.** Soit $D = \{"a" : 2, "b" : 1\}$. Quels programmes affichent "a" et "b" ?

- (A) `for x in D:
 print(x)`      (B) `for x in D.keys():
 print(x)`      (C) `for x in D.items():
 print(x)`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

I.4 – Divers

▷ **Question 17.** Quelles instructions sont correctes, sachant que `sqrt` est dans le module `math` ?

- (A) `from math import sqrt` (B) `from sqrt import math` (C) `import math from sqrt`
 (D) `import sqrt from math` (E) `import math` (F) `import math as m`
 (G) `import * as math` (H) `from math import *` (I) `from math import sqrt as s`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)
 ☐ (F) ☐ (G) ☐ (H) ☐ (I)

▷ **Question 18.** Que représente le type `[(int, int)]` ?

- (A) Un entier. (B) Un couple d'entiers. (C) Une liste d'entiers.
 (D) Une liste de couples d'entiers. (E) Un couple de listes d'entiers.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

Partie II – Algorithmique

II.1 – Recherche dichotomique

▷ **Question 19.** Dans quel(s) objet(s) puis-je faire une recherche dichotomique ?

- (A) Dans une liste triée. (B) Dans une liste quelconque. (C) Dans un dictionnaire Python.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 20.** Quelle est la complexité d'une recherche dichotomique dans L ?

- (A) $O(1)$. (B) $O(\log n)$ avec $n = \text{len}(L)$. (C) $O(n)$ avec $n = \text{len}(L)$.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

II.2 – Algorithmes gloutons

▷ **Question 21.** Quelles propositions sont vraies ?

- (A) Un algorithme glouton permet de trier des données.
- (B) Un algorithme glouton ne remet jamais en cause ses choix précédents.
- (C) Un algorithme glouton effectue toujours le choix localement optimal.
- (D) Un algorithme glouton renvoie toujours la solution optimale.
- (E) Un algorithme glouton a faim.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

II.3 – Matrices

▷ **Question 22.** Quelle instruction crée une matrice de 0 ayant n lignes et m colonnes ?

- (A) `[[0 for i in range(n)] for j in range(m)]`
- (B) `[[0 for i in range(m)] for j in range(n)]`
- (C) `[[0] for i in range(n)] for j in range(m)`
- (D) `[[0] for i in range(m)] for j in range(n)`
- (E) `[[0] * n] * m`
- (F) `[[0] * m] * n`
- (G) `matrix(n, m, 0)`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)
☐ (F) ☐ (G)

▷ **Question 23.** Soit M une liste de listes. Que renvoie l'expression `M.copy()` ?

- (A) Une copie superficielle de M (les lignes de M sont copiées, mais pas les éléments internes).
- (B) Une copie profonde de M (les lignes de M et les éléments internes sont copiés).
- (C) La même matrice que M (rien n'est copié).

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

II.4 – Tris

▷ **Question 24.** Quelle est la complexité pire cas d'un tri par insertion d'une liste, en fonction de sa longueur n ? (A) $O(1)$ (B) $O(\log n)$ (C) $O(n)$ (D) $O(n \log n)$ (E) $O(n^2)$

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

▷ **Question 25.** Quelle est la complexité pire cas d'un tri fusion d'une liste, en fonction de sa longueur n ? (A) $O(1)$ (B) $O(\log n)$ (C) $O(n)$ (D) $O(n \log n)$ (E) $O(n^2)$

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

▷ **Question 26.** Quelle est la complexité pire cas d'un tri rapide d'une liste, en fonction de sa longueur n ? (A) $O(1)$ (B) $O(\log n)$ (C) $O(n)$ (D) $O(n \log n)$ (E) $O(n^2)$

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

▷ **Question 27.** Quelle est la complexité moyenne d'un tri rapide d'une liste, en fonction de sa longueur n ? (A) $O(1)$ (B) $O(\log n)$ (C) $O(n)$ (D) $O(n \log n)$ (E) $O(n^2)$

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

Partie III – Analyse des algorithmiques

III.1 – Signature

▷ **Question 28.** Quelle fonction respecte la signature $f(x:\text{int}, L:[\text{float}]) \rightarrow \{\text{float}:\text{int}\}$?

(A)

```
def f(x, L):
    d = {}
    for y in L:
        d[x] = y
    return d
```

(B)

```
def f(x, L):
    d = {}
    for y in L:
        d[y] = x
    return d
```

(C)

```
def f(x, L):
    d = []
    for y in L:
        d[x] = y
    return d
```

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

III.2 – Correction et terminaison

▷ **Question 29.** Quelles propositions sont vraies?

- (A) Un variant est un entier.
- (B) Un variant peut être un flottant.
- (C) Un variant doit être minoré.
- (D) Un variant doit décroître à chaque itération de la boucle à la quelle il est rattaché.
- (E) Un variant doit décroître strictement à chaque itération de la boucle à la quelle il est rattaché.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

▷ **Question 30.** Quelles propositions sont vraies?

- (A) Un invariant est un entier.
- (B) Un invariant est une propriété (ou un prédicat).
- (C) On doit montrer qu'un invariant est vrai avant la première itération de la boucle.
- (D) On doit montrer que si l'invariant est vrai au début de la boucle, il reste vrai soit à la fin de l'itération, soit au début de l'itération suivante, selon les cas.
- (E) On doit montrer qu'un invariant est vrai après la dernière itération de la boucle.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

▷ **Question 31.** Quelles sont toutes les choses à prouver pour montrer qu'une fonction termine ?

- (A) Toutes les boucles de la fonction ont un variant.
- (B) Toutes les boucles de la fonction ont un invariant.
- (C) Toutes les instructions de la fonction terminent.
- (D) Toutes les fonctions appelées par la fonction terminent.
- (E) La fonction est correcte.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

▷ **Question 32.** Quelles sont toutes les choses à prouver pour montrer qu'une fonction est correcte ?

- (A) Toutes les boucles de la fonction ont un variant.
- (B) Toutes les boucles de la fonction ont un invariant.
- (C) Toutes les fonctions appelées par la fonction sont correctes.
- (D) Toutes les fonctions appelées par la fonction ont leurs préconditions vérifiées.
- (E) La fonction termine.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

III.3 – Complexité

▷ **Question 33.** Quelle est la complexité de l'instruction $L2 = L$ dans le cas où L est une liste de longueur n ? (A) $O(1)$ (B) $O(\log n)$ (C) $O(n)$ (D) $O(n \log n)$ (E) $O(n^2)$

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

▷ **Question 34.** Quelle est la complexité de l'instruction $L2 = L[0:n]$ dans le cas où L est une liste de longueur n ? (A) $O(1)$ (B) $O(\log n)$ (C) $O(n)$ (D) $O(n \log n)$ (E) $O(n^2)$

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

▷ **Question 35.** Quelles instructions ont une complexité constante, où L est une liste ?

- (A) `L.append(0)` (B) `L.pop()` (C) `L.pop(0)` (D) `L.copy()` (E) `L.sort()`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

▷ **Question 36.** Quelles instructions ont une complexité linéaire dans le pire cas, où L est une liste ?

- (A) `L.append(0)` (B) `L.pop()` (C) `L.pop(0)` (D) `L.copy()` (E) `L.sort()`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

Partie IV – Représentation des nombres

IV.1 – Entiers

▷ **Question 37.** Quel est le plus grand entier que l'on peut représenter sur 8 bits, avec une représentation d'entiers non signés ?

- (A) 8 (B) 31 (C) 32 (D) 255 (E) 256

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

▷ **Question 38.** Quel(s) nombre(s) sont représentables en complément à 2 sur 8 bits ?

- (A) -128 (B) -127 (C) 0 (D) 127 (E) 128

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

▷ **Question 39.** Quel(s) entiers(s) sont représentables en Python ?

- (A) 2^{15} (B) 2^{31} (C) 2^{63} (D) $2^{2^{15}}$ (E) $+\infty$

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

IV.2 – Flottants

▷ **Question 40.** Soit x et y deux réels. Soit x le flottant le plus proche de x , et y le flottant le plus proche de y . En choisissant bien x et y , quels tests mathématiquement faux peut-on rendre vrai ?

- (A) $x + 1 == x$ (B) $x != x$ (C) $(x + y) + 1 != x + (y + 1)$ (D) $x - x != 0$

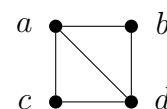
Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

Partie V – Graphes

V.1 – Graphes non-orientés

▷ **Question 41.** Quel est le nombre de sommets de ce graphe ?

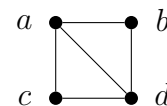
- (A) 1 (B) 4 (C) 5



Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 42.** Quel est le nombre d'arêtes de ce graphe ?

- (A) 1 (B) 4 (C) 5

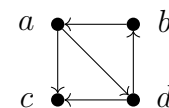


Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

V.2 – Graphes orientés

▷ **Question 43.** Quel est le degré du sommet a ?

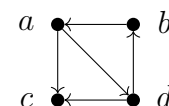
- (A) 1 (B) 2 (C) 3



Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 44.** Quel est le degré entrant du sommet a ?

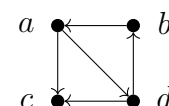
- (A) 1 (B) 2 (C) 3



Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 45.** Quel est le degré sortant du sommet a ?

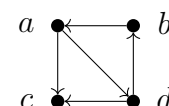
- (A) 1 (B) 2 (C) 3



Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 46.** Quels sont les chemins allant du sommet a au sommet c ?

- (A) $a \rightarrow c$ (B) $a \rightarrow d \rightarrow c$ (C) $a \rightarrow b \rightarrow d \rightarrow c$

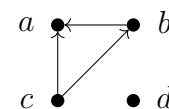


Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

V.3 – Implémentation des graphes

▷ **Question 47.** Quel dictionnaire contient les listes d'adjacence des sommets de ce graphe ?

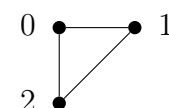
- (A) {'b' : 'a', 'c' : 'a', 'c' : 'b'}
- (B) {'a' : [], 'b' : ['a'], 'c' : ['a', 'b'], 'd' : []}
- (C) {'a' : ['b', 'c'], 'b' : ['c'], 'c' : [], 'd' : []}



Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 48.** Quelle liste contient les listes d'adjacence des sommets de ce graphe ?

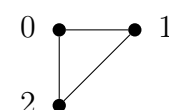
- (A) [0, 1, 2]
- (B) [[0], [1], [2]]
- (C) [[1, 2], [[0, 2], [[0, 1]]]
- (D) [[0, 1, 2], [[0, 1, 2], [[0, 1, 2]]]



Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

▷ **Question 49.** Quelle est la matrice d'adjacence de ce graphe ?

- (A) $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ (B) $\begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$ (C) $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$



Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

V.4 – Parcours de graphe

▷ **Question 50.** Quelles propositions sont vraies ?

- (A) Un parcours en profondeur s'effectue au moyen d'une pile.
- (B) Un parcours en profondeur s'effectue au moyen d'une file.
- (C) Un parcours en largeur s'effectue au moyen d'une pile.
- (D) Un parcours en largeur s'effectue au moyen d'une file.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)
--

▷ **Question 51.** À quoi cela sert-il de parcourir un graphe ?

- (A) Cela permet de visiter les sommets du graphe.
- (B) Cela permet de savoir si un graphe est connexe.
- (C) Cela permet de savoir si un graphe contient un cycle.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

Partie VI – Dictionnaires et programmation dynamique

VI.1 – Dictionnaires

▷ **Question 52.** Quel type permet d'implémenter une table de hachage (avec adressage fermé) pour un dictionnaire de type `{str:float}` ?

- (A) `[str]` (B) `[float]` (C) `[(str, float)]` (D) `[[ (str, float) ]]`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)
--

▷ **Question 53.** Quels types peuvent être clés d'un dictionnaire ?

- (A) `float` (B) `str` (C) `(int, int)` (D) `[(int, int)]` (E) `{int:int}`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

▷ **Question 54.** Quels types peuvent être valeurs d'un dictionnaire ?

- (A) `float` (B) `str` (C) `(int, int)` (D) `[(int, int)]` (E) `{int:int}`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

▷ **Question 55.** Quelles instructions s'effectuent en temps moyen constant dans un dictionnaire d'implémenté par une table de hachage ?

- (A) `d[x] = y` (B) `d[x] == y` (C) `x in d` (D) `x in d.items()`

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)
--

VI.2 – Programmation dynamique

▷ **Question 56.** Qu'est-ce que la mémoïsation ?

- (A) C'est le fait d'essayer toutes les solutions possibles pour résoudre un problème.
- (B) C'est le fait de stocker les calculs déjà effectués, afin de ne pas les refaire.
- (C) C'est le fait de programmer en utilisant des fonctions récursives.
- (D) C'est de la mémorisation, mais sans *r*.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

▷ **Question 57.** Quelles structures peuvent être utilisées pour implémenter la mémoïsation ?

- (A) Des dictionnaires.
- (B) Des listes.
- (C) Des tuples.
- (D) Des chaînes de caractères.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

▷ **Question 58.** Qu'est-ce qui est vrai à propos de la programmation dynamique ?

- (A) On essaie toutes les solutions possibles pour résoudre un problème.
- (B) On découpe un problème en sous-problème(s) plus petit(s).
- (C) On l'utilise quand certains sous-problèmes se chevauchent, c'est-à-dire que l'on retrouve le même sous-problème plusieurs fois.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 59.** Le code ci-contre contient un exemple d'une fonction résolvant un problème par programmation dynamique. Que doit-on faire dans les ... ?

- (A) En général, calculer `aux(i)` via des appels récursifs.
- (B) Ne pas recalculer `aux(i)` si *i* est une clé de `memo`.
- (C) Stocker le résultat de `aux(i)` dans `memo[i]`.

```
def prog_dyn(L):  
    memo = {}  
    def aux(i):  
        ...  
    return aux(len(L))
```

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 60.** En général, quelle est la complexité d'un algorithme de programmation dynamique qui utilise un dictionnaire `memo` pour effectuer sa mémoïsation ainsi qu'une fonction auxiliaire récursive `aux` pour remplir le dictionnaire `memo` ?

- (A) C'est le nombre de clés dans `memo` plus la complexité de `aux`.
- (B) C'est le nombre de clés dans `memo` plus la complexité propre de `aux`, c'est-à-dire sans compter les appels récursifs.
- (C) C'est le nombre de clés dans `memo` multiplié par la complexité de `aux`.
- (D) C'est le nombre de clés dans `memo` multiplié par la complexité propre de `aux`, c'est-à-dire sans compter les appels récursifs.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

Partie VII – Bases de données

VII.1 – Vocabulaire

On considère la table **Musique** ci-contre.

Musique			
<i>Titre</i>	<i>Artiste</i>	<i>BPM</i>	<i>Tonalité</i>
Telephone	Lady Gaga	122	Fm
Levels	Avicii	126	E
Titanium	David Guetta	126	Cm

▷ **Question 61.** Donnez un synonyme de « table ».

- (A) « relation » (B) « modèle » (C) « schéma » (D) « données »

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

▷ **Question 62.** Donnez un attribut de la table **Musique**.

- (A) *Titre*. (B) Telephone. (C) (Levels, Avicii, 126, E). (D) (*Titre*, *Artiste*, *BPM*, *Tonalité*).

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

▷ **Question 63.** Donnez un enregistrement de la table **Musique**.

- (A) *Titre*. (B) Telephone. (C) (Levels, Avicii, 126, E). (D) (*Titre*, *Artiste*, *BPM*, *Tonalité*).

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

▷ **Question 64.** Donnez le schéma de la table **Musique**.

- (A) *Titre*. (B) Telephone. (C) (Levels, Avicii, 126, E). (D) (*Titre*, *Artiste*, *BPM*, *Tonalité*).

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

▷ **Question 65.** Quels types de données sont acceptables pour la colonne *BPM* de la table **Musique** ?

- (A) Entier. (B) Flottant. (C) Chaîne de caractères.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 66.** Quelle(s) condition(s) doit vérifier une clé primaire ?

- (A) Être associée à un unique attribut.
 (B) Être différente pour chaque ligne de la table.
 (C) Être un entier.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 67.** Quelle(s) condition(s) doit vérifier une clé étrangère ?

- (A) Être associée à un unique attribut.
 (B) Être différente pour chaque ligne de la table.
 (C) Faire référence à une clé primaire.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

VII.2 – Langage SQL

▷ **Question 68.** Cochez les mots-clés SQL que vous devez connaître (car ils sont au programme).

(A) SELECT (B) WHERE (C) WHEN (D) AS (E) DIFFERENT (F) DISTINCT
 (G) LIMIT (H) OFF (I) OFFSET (J) SORT BY (K) ORDER BY (L) UNION
 (M) INTERSECTION (N) INTERSECT (O) EXCEPT (P) MINUS (Q) JOIN ... ON
 (R) COUNT (S) MEAN (T) AVG (U) GROUP BY (V) GROUPED BY (W) HAVING

Réponse(s) :	☐ (A)	☐ (B)	☐ (C)	☐ (D)	☐ (E)
	☐ (F)	☐ (G)	☐ (H)	☐ (I)	☐ (J)
	☐ (K)	☐ (L)	☐ (M)	☐ (N)	☐ (O)
	☐ (P)	☐ (Q)	☐ (R)	☐ (S)	☐ (T)
	☐ (U)	☐ (V)	☐ (W)		

▷ **Question 69.** Quelle commande SQL permet d'effectuer le test « A_1 = A_2 » ?

(A) A_1 = A_2 (B) A_1 == A_2 (C) A_1 === A_2 (D) A_1 EQUAL A_2

Réponse(s) :	☐ (A)	☐ (B)	☐ (C)	☐ (D)
--------------	------------------------------	------------------------------	------------------------------	------------------------------

▷ **Question 70.** Quelle commande SQL permet d'effectuer le test « A_1 ≠ A_2 » ?

(A) A_1 != A_2 (B) A_1 ≠ A_2 (C) A_1 <> A_2 (D) A_1 NOT A_2

Réponse(s) :	☐ (A)	☐ (B)	☐ (C)	☐ (D)
--------------	------------------------------	------------------------------	------------------------------	------------------------------

▷ **Question 71.** Si A_0 est un attribut, à quoi sert GROUP BY A_0 en SQL ?

- (A) À sélectionner la colonne A_0.
 (B) À regrouper ensemble les lignes qui ont la même valeur pour l'attribut A_0.
 (C) À trier les lignes suivant l'attribut A_0.

Réponse(s) :	☐ (A)	☐ (B)	☐ (C)
--------------	------------------------------	------------------------------	------------------------------

▷ **Question 72.** Je souhaite supprimer des lignes dans une requête SQL dans laquelle j'utilise GROUP BY. Quelles propositions sont vraies ?

- (A) J'utilise WHERE pour supprimer des lignes *avant* la fusion des lignes par GROUP BY.
 (B) J'utilise HAVING pour supprimer des lignes *avant* la fusion des lignes par GROUP BY.
 (C) J'utilise WHERE pour supprimer des lignes *après* la fusion des lignes par GROUP BY.
 (D) J'utilise HAVING pour supprimer des lignes *après* la fusion des lignes par GROUP BY.
 (E) Je ne peux pas utiliser à la fois WHERE et HAVING dans une même requête.

Réponse(s) :	☐ (A)	☐ (B)	☐ (C)	☐ (D)	☐ (E)
--------------	------------------------------	------------------------------	------------------------------	------------------------------	------------------------------

Partie VIII – Intelligence artificielle et théorie des jeux

VIII.1 – Intelligence artificielle

▷ **Question 73.** J'ai une liste d'environ un milliard d'images, qui représentent chacune soit un chat, soit un chien. Je veux séparer ces images en deux groupes : celles qui représentent un chat et celles qui représentent un chien. Que puis-je faire ?

- (A) Je le fais manuellement en regardant toutes les images, ce qui me prendra environ un an.
- (B) Je le fais manuellement en regardant toutes les images, ce qui me prendra environ un siècle.
- (C) J'utilise l'algorithme des k plus proches voisins.
- (D) J'utilise l'algorithme des k -moyennes.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

▷ **Question 74.** J'ai une liste d'environ un milliard d'images, qui représentent chacune soit un chat, soit un chien. Je sais pour chaque image si elle représente un chat ou un chien. On me donne une nouvelle liste d'environ un milliard d'images qui représentent aussi un chat ou un chien. Je veux savoir, pour chacune de ces nouvelles images, si c'est un chat ou un chien. Que puis-je faire ?

- (A) J'utilise l'algorithme des k plus proches voisins.
- (B) J'utilise l'algorithme des k -moyennes.
- (C) Plus généralement, j'utilise un algorithme d'apprentissage supervisé.
- (D) Plus généralement, j'utilise un algorithme d'apprentissage non-supervisé.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

▷ **Question 75.** Je veux savoir si mon algorithme qui sépare les images de chat et les images de chien par intelligence artificielle donne de bons résultats. Quel outil puis-je utiliser ?

- (A) Une matrice de confusion.
- (B) Une matrice d'attracteurs.
- (C) Une matrice min-max.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

VIII.2 – Théorie des jeux

▷ **Question 76.** Comment avons-nous décidé de modéliser un jeu à 2 joueurs ?

- (A) Par un arbre.
- (B) Par un graphe biparti.
- (C) Par un dictionnaire.
- (D) Par un plateau.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

▷ **Question 77.** Quel(s) jeu(x) à 2 joueurs peuvent-être modélisés de cette manière ?

- (A) Les échecs.
- (B) Le Uno.
- (C) Le Go.
- (D) Le Monopoly.
- (E) Le Puissance 4.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D) ☐ (E)

▷ **Question 78.** En général, qu'est-ce qu'un état dans un jeu ?

- (A) C'est la donnée du plateau de jeu.
- (B) C'est la donnée du plateau de jeu et du joueur qui doit jouer.
- (C) C'est la donnée du plateau de jeu, du joueur qui doit jouer et de tous les coups possibles.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 79.** Puisqu'un jeu est modélisé par un graphe biparti, qu'est-ce qu'un état final ?

- (A) L'ensemble des états finals est un ensemble quelconque d'états choisis à l'avance.
- (B) Un état est final si, et seulement si, il n'y a pas d'arête sortant de cet état.
- (C) Un état est final si, et seulement si, il n'y a pas d'arête entrant dans cet état.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 80.** Les états finals d'un jeu sont partitionnés en trois : les états gagnants pour J_1 , les états gagnants pour J_2 et les états de match nul. Comment cette partition est-elle déterminée ?

- (A) Elle est déterminée à l'avance et selon les règles du jeu.
- (B) Un état final est gagnant pour J_1 si, et seulement si, il est contrôlé par J_1 . Idem pour J_2 .
- (C) Un état final est gagnant pour J_1 si, et seulement si, il est contrôlé par J_2 , et vice-versa.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 81.** En théorie des jeux, qu'est-ce qu'une stratégie ?

- (A) Un ensemble d'états.
- (B) Une fonction mathématique de l'ensemble des états vers l'ensemble des états.
- (C) Une fonction Python prenant un état et renvoyant une liste d'états.
- (D) Une liste de coups (qui mènent à la victoire dans le cas où je veux une stratégie gagnante).

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

▷ **Question 82.** Qu'est-ce que l'attracteur d'un joueur ?

- (A) L'ensemble des états qui sont gagnants pour ce joueur.
- (B) L'ensemble des états à partir desquels ce joueur peut atteindre un état final.
- (C) L'ensemble des états à partir desquels ce joueur a une stratégie gagnante.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 83.** Comment peut-on déterminer une stratégie gagnante pour un joueur ?

- (A) À partir de l'ensemble des attracteurs.
- (B) Via l'algorithme min-max.
- (C) Via l'algorithme min-max muni d'une heuristique. Dans ce cas, la stratégie n'est pas forcément gagnante ; il s'agit d'une approximation.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

Partie IX – Analyse de documents

IX.1 – Manipulation de fichiers

On donne la documentation des fonctions permettant de lire un fichier texte :

- L'instruction `f = open(nom, mode)`, où `nom` et `mode` sont des chaînes de caractères, ouvre un fichier nommé `nom`. La variable `mode` indique comment le fichier sera utilisé : `'r'` pour uniquement lire le fichier et `'r+'` pour lire et écrire dans le fichier. La variable `f` représente une position dans ce fichier. Initialement, la position est au début du fichier.
- L'instruction `f.read()` renvoie une chaîne de caractères contenant le texte qui se situe entre la position représentée par `f` et la fin du fichier. La variable `f` représente maintenant la position à la fin du fichier.
- L'instruction `f.readlines()` renvoie une liste de chaînes de caractères. Chaque ligne du fichier est placée dans une case différente de la liste renvoyée. Seul le texte qui se situe entre la position représentée par `f` et la fin du fichier est pris en compte. La variable `f` représente maintenant la position à la fin du fichier.
- L'instruction `f.readline()` renvoie une chaînes de caractères contenant le texte qui se situe entre la position représentée par `f` et la fin de la ligne. La variable `f` représente maintenant la position au début de la ligne suivante. Si `f` représente la fin du fichier, la chaîne vide est renvoyée. Sinon, la chaîne contient au moins le caractère `'\n'` de fin de ligne.
- L'instruction `f.close()` ferme le fichier.

▷ **Question 84.** Quelles instructions permettent d'afficher le fichier `'a.txt'` ?

- | | | | |
|---|--|--|---|
| <p>(A) <pre>f = open('a.txt', 'r+') print(f.read()) f.close()</pre></p> | <p>(C) <pre>f = open('a.txt', 'r') x = '' while x != '': x = readline() print(x) f.close()</pre></p> | <p>(D) <pre>f = open('a.txt', 'r') x = '\n' while x != '': x = readline() print(x) f.close()</pre></p> | <p>(B) <pre>f = open('a.txt', 'w') print(f.readlines()) f.close()</pre></p> |
|---|--|--|---|

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

On donne la documentation des fonctions permettant d'écrire dans un fichier texte :

- L'instruction `f = open(nom, mode)`, où `nom` et `mode` sont des chaînes de caractères, ouvre un fichier nommé `nom`. La variable `mode` indique comment le fichier sera utilisé : `'w'` et `'a'` pour uniquement écrire dans le fichier et `'r+'` pour lire et écrire dans le fichier. La variable `f` représente une position dans ce fichier. Initialement, la position est au début du fichier, sauf si `mode` vaut `'a'`, où elle est dans ce cas à la fin du fichier.
- L'instruction `f.write(x)` écrit la chaîne de caractères `x` à la position représentée par `f`. La variable `f` représente maintenant la position à la fin de `x`.

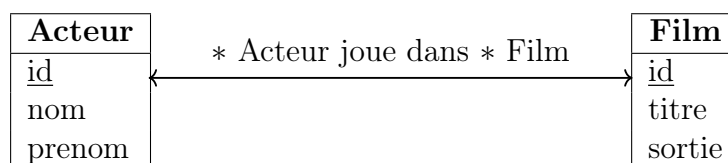
▷ **Question 85.** Quelle instruction permet d'écrire `'fin'` à la fin du fichier `'a.txt'` ?

- | | | |
|---|---|--|
| <p>(A) <pre>f = open('a.txt', 'w') f.write('fin') f.close()</pre></p> | <p>(B) <pre>f = open('a.txt', 'a') f.write('fin') f.close()</pre></p> | <p>(C) <pre>f = open('a.txt', 'r+') f.write('fin') f.close()</pre></p> |
|---|---|--|

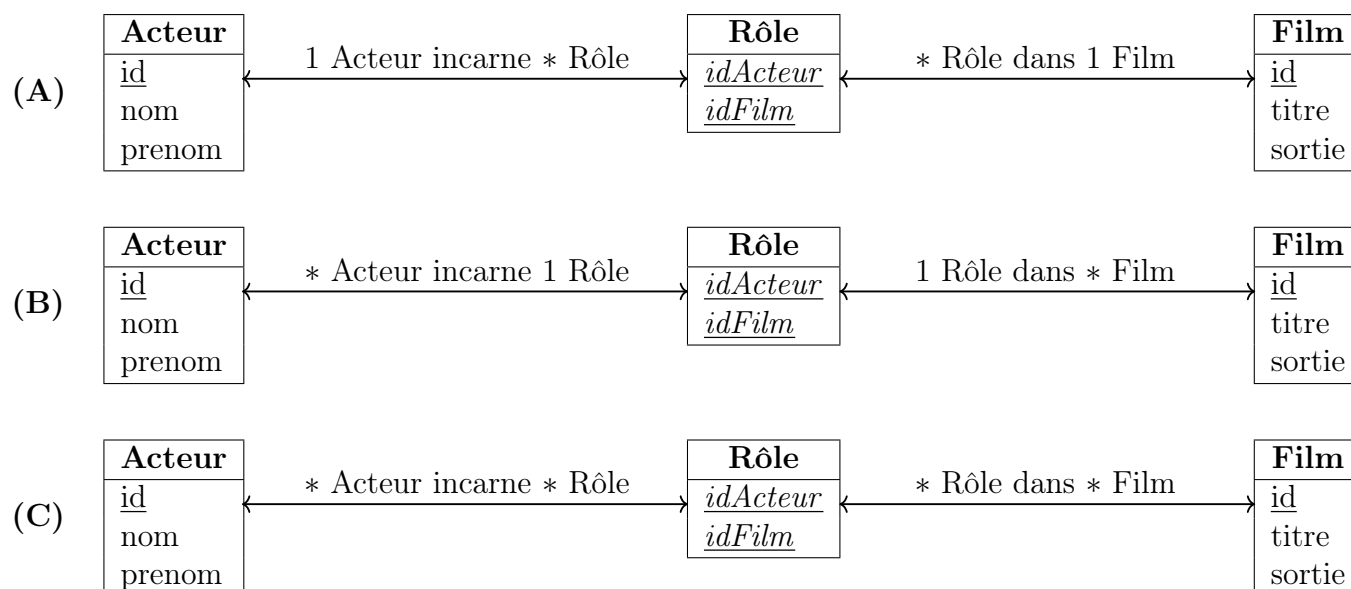
Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

IX.2 – Modèle entité-association

On considère le schéma suivant :

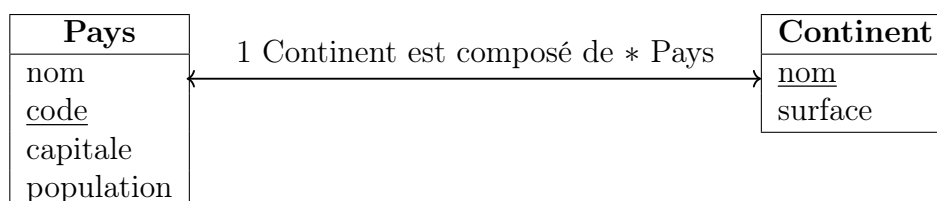


▷ **Question 86.** Quel schéma, équivalent au schéma ci-dessus, aide à traduire en SQL la relation $* - *$ présente entre les entités **Acteur** et **Film** ?



Réponse(s) : ☐ (A) ☐ (B) ☐ (C)

▷ **Question 87.** Comment traduit-on en SQL une relation $1 - *$, comme celle du schéma ci-dessous ?



- (A) Au moyen d'une clé *étrangère* dans la table **Pays** et d'une clé *étrangère* dans la table **Continent**.
 (B) Au moyen d'une clé *primaire* dans la table **Pays** et d'une clé *étrangère* dans la table **Continent**.
 (C) Au moyen d'une clé *étrangère* dans la table **Pays** et d'une clé *primaire* dans la table **Continent**.
 (D) Au moyen d'une clé *primaire* dans la table **Pays** et d'une clé *primaire* dans la table **Continent**.

Réponse(s) : ☐ (A) ☐ (B) ☐ (C) ☐ (D)

Fin des révisions.